

PharmaGen

Feature & Implementation Inventory

Precision oncology platform · FastAPI backend + Streamlit frontend
Generated from a full repository read-through · August 2026

Contents

1	Overview	1
2	Backend API layer	2
2.1	Application setup and middleware	2
2.2	Endpoint: GET /health	2
2.3	Endpoint: POST /api/v1/analyze	2
3	Core services	2
3.1	VCF parsing & evidence matching engine	2
3.2	Interactive knowledge graph generator	4
3.3	HTML report builder	4
3.4	PDF report builder	4
4	Knowledge base layer	4
4.1	Schema creation and migration	4
4.2	CIViC ingest (primary source)	5
4.3	OncoKB ingest (hybrid second source, independent of CIViC success)	5
5	Frontend (Streamlit console)	5
5.1	Connectivity and session management	5
5.2	Visual identity	5
5.3	Result presentation	5
5.4	Tabs	6
5.5	Landing state	6
6	CLI data tooling	6
7	Test suite	7
8	Developer tooling & infrastructure	7
8.1	Make targets	7
8.2	Launch scripts	7
8.3	Container support	7
8.4	Quality gates	7
8.5	Repository hygiene files	8
9	Environment variable reference	8
10	Cross-cutting policies	8

1 Overview

PharmaGen ingests VCF (Variant Call Format) genomic files, extracts gene and mutation calls, matches them against a local SQLite clinical knowledge base built from CIViC and OncoKB evidence, and presents the results as an actionable treatment matrix, an interactive knowledge graph, and downloadable reports.

Core flow:

VCF upload →	parse_vcf_stream_detailed()	backend/app/services/vcf_parser.py
→	match_clinical_evidence()	backend/app/services/vcf_parser.py
→	/api/v1/analyze JSON response	backend/app/main.py
→	Streamlit matrix + graph + AI UI	frontend/app.py

Two-package layout: the backend is served with uvicorn using `--app-dir backend`, so application code imports as `from app.services...`; the test suite imports as `from backend.app.services...`

2 Backend API layer

File: backend/app/main.py — FastAPI application (app), all HTTP endpoints.

2.1 Application setup and middleware

- **FastAPI instance** titled “PharmaGen Clinical API” — backend/app/main.py.
- **CORS middleware**: allows all origins, methods, and headers, so the Streamlit frontend can call the API from any host — backend/app/main.py.
- **Startup database index**: create_db_indices() runs once on startup and issues CREATE INDEX IF NOT EXISTS idx_variant_evidence_upper_gene_mut on (UPPER(gene), UPPER(mutation)), making case-insensitive evidence lookups fast; failures are printed as warnings and never block startup — backend/app/main.py.
- **Upload size guard**: constant MAX_UPLOAD_BYTES = 10 MiB bounds accepted VCF uploads for a fast, deterministic request path — backend/app/main.py.

2.2 Endpoint: GET /health

Health probe reporting service liveness plus knowledge-base state. Returns one of three knowledge-base states: missing (DB file absent), empty (table missing, sqlite3.OperationalError), or loaded, each with an evidence_records count from SELECT COUNT(*). The Docker health check and the frontend connection probe both consume this endpoint — backend/app/main.py.

2.3 Endpoint: POST /api/v1/analyze

The main analysis pipeline. Accepts a multipart VCF upload (UploadFile) and returns the full annotation payload.

Input guards (in order):

1. **400** if the uploaded content is empty or whitespace only.
2. **413** if larger than MAX_UPLOAD_BYTES (10 MB).
3. **400** if no variants parsed and zero data rows were seen (bad headers/empty body).
4. **503** if the knowledge base file does not exist yet, with remediation hint “Run: make bootstrap”.

Processing steps:

- Parses bytes via parse_vcf_stream_detailed (validation report included in response).
- **Per-key query deduplication**: builds a unique (gene.upper(), mutation.upper()) key set so each distinct variant hits SQLite exactly once even when it appears in many rows/patients; results are reused for repeats — implemented inline in analyze_patient_vcf, backend/app/main.py.
- Tallies exact_matches, contextual_matches, and no_matches by scanning each variant’s matches for match_type.
- **Synthetic-data detection**: scans the first 8192 bytes for the markers ##synthetic_data=true or SYNTHETIC=1 and returns a boolean flag used by the frontend to label demo data.

Response JSON keys: status, variants_count, unique_genes, exact_matches, contextual_matches, no_matches, synthetic_data, input_validation, annotated_results (each item is {variant_info, clinical_matches}) — all in backend/app/main.py.

3 Core services

All services live under backend/app/services/.

3.1 VCF parsing & evidence matching engine

File: backend/app/services/vcf_parser.py — class VariantAnnotationEngine (all static methods).

3.1.1 Knowledge-base path resolution

DB_PATH is derived from the module location (backend/data/raw/clinical_kb.db), so the parser works regardless of the process working directory — backend/app/services/vcf_parser.py.

3.1.2 Mutation normalization tables

- **Amino-acid map** HGVS_MAP: translates all 20 three-letter amino acid codes to one-letter codes — vcf_parser.py.
- **Position-anchored regex** _AA3_RE: converts a three-letter code only when followed by a digit or end-of-string, with a letter-excluding lookbehind, so p.Val600Glu becomes V600E while ordinary words are untouched — vcf_parser.py.

- **Known rsID map** `RSID_MAP`: maps pharmacogenomic rsIDs directly to (gene, mutation) pairs: rs4244285 → CYP2C19 *2, rs3918290 → DPYD *2A, rs9923231 → VKORC1 Warfarin, rs121913333 → BRAF V600E — `vcf_parser.py`.

3.1.3 Parser: `parse_vcf_stream_detailed(file_bytes)`

Returns (variants, validation_report); this is the only sanctioned ingestion path (the frontend quality report depends on it).

Header handling:

- Recognizes `##fileformat=VCF...` (sets `fileformat_header`) and `#CHROM...` column header (sets `column_header`); other `#` lines ignored; blank lines skipped.
- Data rows must have at least 8 tab-separated columns, else they increment `skipped_rows`.
- INFO fields are split on `;` into an uppercase-keyed dict.

Gene/mutation extraction cascade (first source that yields a value wins; later sources still override when earlier ones left gaps per the code order):

1. Custom PharmaGen INFO keys: GENE or SYMBOL for gene; MUT or HGVSP for mutation.
2. SnpEff ANN= field: gene at pipe-index 3, HGVSp at index 10.
3. VEP CSQ= field: same indices.
4. Known rsID override via `RSID_MAP` when the ID column starts with rs.
5. Fallback mutation string REF>ALT when nothing else was found.

Mutation normalization applied before storage/matching:

1. Strip spaces.
2. Strip transcript prefixes by taking the substring after the last `:` (e.g., ENSP00000288602:p.Val600Glu).
3. Strip leading protein/coding change prefixes `p.` / `c..`
4. Translate three-letter amino acids to one-letter codes via the anchored regex.

Cohort-aware duplicate detection:

1. Dedup key is (chrom, pos, ref, alt, gene.upper(), mutation.upper(), sample_id) where sample_id comes from the INFO SAMPLE key.
2. The same hotspot appearing in **different** patients is kept as distinct instances; only a repeat within the same patient counts as a duplicate (`duplicate_rows`).

Validation report keys produced: `fileformat_header`, `column_header`, `data_rows`, `parsed_rows`, `skipped_rows`, `gene_annotated_rows`, `mutation_annotated_rows`, `duplicate_rows`, `valid_vcf_headers` (both headers present), `annotation_coverage_percent` (share of parsed rows with a real gene), plus cohort stats `patients_observed` (distinct SAMPLE values) and `unique_variant_combinations`.

Stored variant dicts contain exactly chrom, pos, gene, mutation, with gene/mutation uppercased — all in `backend/app/services/vcf_parser.py`.

3.1.4 Deprecated wrapper: `parse_vcf_stream(file_bytes)`

Legacy convenience wrapper returning only the variant list; internally delegates to the detailed parser — `backend/app/services/vcf_parser.py`.

3.1.5 Evidence matcher: `match_clinical_evidence(gene, mutation, cursor=None)`

Queries the knowledge base and classifies every lookup into one of three match types: "exact", "gene_context", "none".

Implementation details:

- Normalizes the incoming mutation identically to the parser (prefix/transcript stripping), so raw HGVS input hits the KB directly.
- **Optional shared cursor**: opens (and owns) its own SQLite connection when no cursor is passed; reuses the caller's cursor otherwise, enabling one-query-per-unique-variant batching in `/api/v1/analyze`.
- **Column-name introspection**: runs `PRAGMA table_info(variant_evidence)` and selects the mutation column dynamically among mutation, variant, alteration — never hardcoded (banned-pattern rule).

- **Exact query:** DISTINCT disease, therapy, evidence_tier, source where gene matches case-insensitively and the mutation equals either the original or cleaned form.
- **Contextual fallback:** when no exact rows exist, fetches up to 10 rows for the same gene and labels them match_type = "gene_context".
- **None sentinel:** unknown gene+mutation yields a single placeholder record ("No Direct Match", tier "Unclassified") with match_type = "none" so callers always receive a shaped list.
- Result deduplication preserves first-seen order.

Return record keys: disease, therapy, evidence_tier, source, match_type — all in backend/app/services/vcf_parser.py.

3.2 Interactive knowledge graph generator

File: backend/app/services/graph_engine.py — class KnowledgeGraphService.

generate_interactive_html(annotated_results, output_html_path="frontend/graph.html") builds a PyVis network and writes a standalone HTML file, returning the path.

- Directed graph on a dark background (#0e1117) sized 500 px.
- Fixed color schema: Gene #FF4B4B (ellipse), Mutation #FFAA00 (diamond), Disease #00C0F2 (box), Drug #00D47E (star).
- Edge semantics: Gene --HAS_MUTATION--> Mutation; Mutation --INDICATES--> Disease; Mutation --<tier--> Drug (edge labeled with the evidence level).
- Caps rendering at the **first 10 annotated records** and the **top 3 clinical matches per record**, skipping sentinel "No Direct Match" entries.
- Physics configured with Barnes-Hut parameters (gravitationalConstant -8000, springLength 120); creates output directories as needed — all in backend/app/services/graph_engine.py.

Note: the frontend renders its own focused single-path graph in memory rather than calling this service (see Frontend section); both share the node-type color idea but use different palettes.

3.3 HTML report builder

File: backend/app/services/report_generator.py — function generate_html_report(filename, analysis, rows).

Produces a self-contained styled HTML string for the clinical review document:

- Prominent disclaimer notice block ("not a diagnosis or treatment recommendation").
- Metric grid summarizing variants reviewed, exact matches, contextual variants, unmatched variants.
- Input-validation summary line (valid headers, parsed/skipped/duplicate rows, annotation coverage).
- Two separated tables: **exact clinical evidence** vs **contextual evidence only**, with an explicit warning that gene-only records are not treatment recommendations.
- All cell values escaped via html.escape before interpolation — backend/app/services/report_generator.py.

3.4 PDF report builder

File: backend/app/services/pdf_generator.py — class PDFReportService.

create_clinical_pdf(dataframe) -> bytes builds an in-memory ReportLab document:

- US Letter page with 30 pt margins; title paragraph "PharmaGen Precision Oncology Clinical Report".
- Evidence table limited to the first 15 dataframe rows with columns Gene, Mutation, Disease (truncated to 25 chars), Targeted Drug (truncated to 30 chars), Evidence Level.
- Styled header row (dark background, white bold text) and light grid lines; returns the PDF bytes from the BytesIO buffer — backend/app/services/pdf_generator.py.

4 Knowledge base layer

File: backend/app/db/bootstrap.py — function init_real_civic_db() plus OncoKB loader _load_oncokb(cursor); runnable as python -m backend.app.db.bootstrap.

4.1 Schema creation and migration

- Creates backend/data/raw/ if needed and table variant_evidence with composite primary key (gene, mutation, therapy, disease) and columns gene, mutation, disease, therapy, evidence_tier, source.

- Idempotent migration: checks PRAGMA table_info and adds an adverse_effects TEXT column when absent — bootstrap.py.

4.2 CIViC ingest (primary source)

- Downloads the live CIViC nightly TSV (2500 evidence records) with pandas directly from civicdb.org.
- Dynamic schema mapping handles both modern and legacy CIViC layouts by lowercased column lookup: disease|disease_name; therapy therapies|drugs|therapy; level evidence_level|evidence_direction; variant molecular_profile|variant; gene gene|feature_name.
- Modern molecular-profile parsing splits strings like BRAF V600E on the first space into gene + mutation; legacy branch reads explicit gene and variant columns.
- Evidence levels normalized to Level X strings, defaulting to Level A when absent; rows inserted with INSERT OR REPLACE (CIViC wins conflicts).
- Network/schema failure fallback: inserts a curated 4-record panel — BRAF V600E/Melanoma/Vemurafenib, EGFR L858R/NSCLC/Osimertinib, KRAS G12C/NSCLC/Sotorasib, ERBB2 AMPLIFICATION/Breast Cancer/Trastuzumab — so the platform stays demonstrable offline — bootstrap.py.

4.3 OncoKB ingest (hybrid second source, independent of CIViC success)

Runs after CIViC inside its own try/except, so a CIViC network failure cannot block it.

- **Full dataset discovery:** looks for the GENIE annotation TSV in priority order — PHARMAGEN_ONCOKB_FILE env var, repo root, then tests/ directory.
- **Full TSV parsing:** drops rows lacking CANCER_TYPE_DETAILED; iterates the four level columns; extracts Drug(GENE MUTation) patterns via regex; strips P./C. mutation prefixes; splits semicolon-joined alterations and comma-joined drug names into individual records tagged source = "OncoKB".
- **Tier crosswalk:** OncoKB therapeutic levels mapped onto the CIViC scale so downstream filtering is uniform — LEVEL_1→Level A, LEVEL_2→Level B, LEVEL_3A/3B→Level C, LEVEL_4→Level D.
- **Seed panel fallback:** without the GENIE file, loads the committed 27-row curated panel backend/data/seed/oncokb_seed.csv (standard-of-care Level A/B pairings, including biomarker-style keys such as BRCA1 PATHOGENIC, MET EXON 14 SKIPPING, NTRK/RET/ROS1 fusions, MSI HIGH, TMB HIGH).
- **Conflict policy:** all OncoKB inserts use INSERT OR IGNORE, so existing CIViC evidence for the same primary key is never overwritten — all in bootstrap.py and backend/data/seed/oncokb_seed.csv.

5 Frontend (Streamlit console)

File: frontend/app.py — single-file Streamlit application (590 lines).

5.1 Connectivity and session management

- **API base resolution:** candidate list from PHARMAGEN_API_URL env override plus http://127.0.0.1:8000; probes each candidate's /health with a 1.5 s timeout until one answers — resolve_api_base_url(), frontend/app.py.
- **Backend status strip:** sidebar panel showing connected URL and formatted evidence-record count, or a neutral “not connected” card — frontend/app.py.
- **Upload handling:** sidebar uploader accepting .vcf/.txt; content persisted in session state keyed by SHA-256 hash, so re-uploading the identical file does not re-analyze, while any changed file invalidates cached analysis data and cached AI results — frontend/app.py.
- **Analysis caching:** POSTs the multipart file to /api/v1/analyze under a spinner (“Executing Deterministic Evidence Lookup..”); caches both parsed JSON and raw response bytes in session state for repeat renders and downloads — frontend/app.py.
- Unreachable-backend and non-200 responses surface st.error messages and stop execution cleanly.

5.2 Visual identity

Custom CSS theme injected with unsafe_allow_html: DM Sans + Space Grotesk web fonts, teal/coral palette, dark sidebar, styled metric cards, tabs, dataframes, download buttons, status strips, responsive breakpoint below 900 px width; hero header with brand kicker, headline, accent rule, and tagline — frontend/app.py.

5.3 Result presentation

Status banners chosen by outcome: info banner when synthetic markers were detected; success banner with exact-match count; warning banners when only contextual or no evidence exists — frontend/app.py.

Evidence matrix construction flattens `annotated_results` into one row per clinical match with columns Gene, Mutation, Chromosome, Disease, Targeted Drug, Evidence Level, Source, Match Type — `frontend/app.py`.

Cohort aggregation `_aggregate_cohort(frame)`: groups identical evidence paths across patients into a single row with a Patients counter, sorted by frequency — the same hotspot in N patients collapses to one row with Patients=N — `frontend/app.py`.

Sidebar filters: multiselects for **Evidence Tiers** and **Evidence Sources**; the filtered frame is split into exact-only (`filtered_df`) and gene_context-only (`contextual_df`), each cohort-aggregated separately — `frontend/app.py`.

Overview metrics row: Variants reviewed (with patients delta when cohort >1), Exact Clinical Matches, High-confidence exact (counting **unique** Level A/B biomarkers, not evidence rows), Unique genes; plus a coverage caption line — `frontend/app.py`.

Cohort snapshot expander: when more than one patient was observed, bar chart of the top 12 most frequently altered genes (unique biomarker counts) — `frontend/app.py`.

Input quality expander: five metrics (valid headers, rows parsed, skipped, duplicates, annotation coverage %) plus a statement of the evidence match policy — `frontend/app.py`.

5.4 Tabs

Tab 1 — Actionable Treatment Matrix:

- Exact matches rendered as a cohort dataframe; contextual records isolated behind a collapsible expander carrying an explicit warning that gene-only evidence is not a treatment recommendation.
- Downloads: filtered matrix CSV, full raw analysis JSON, and a client-side assembled HTML clinical review report (inline-styled, exact vs contextual sections, disclaimers, filename echo) — `frontend/app.py`.

Tab 2 — Interactive Knowledge Graph:

- Biomarker selectbox listing exact-matched GENE · MUTATION pairs restricted to the selected tiers; best match chosen after sorting by tier, disease, therapy.
- “Why this result appeared” explanation line tying the selection back to the exact match, tier, and source.
- **Focused PyVis graph** built in memory: hierarchical left-to-right layout, physics disabled, hover and navigation buttons enabled, themed palette (#E76F51 gene ellipse → #D29A31 mutation diamond → #62B7B0 disease box → #8CE0C3 drug star) chained Gene→Mutation→Disease→Drug; rendered container-safely via `components.html(net.generate_html(notebook=False))` with no disk writes — `frontend/app.py`.

5.5 Landing state

When no file is active: “Awaiting genomic input” strip and a three-column introduction (01 Variant scan / 02 Evidence match / 03 Explainable graph) — `frontend/app.py`.

6 CLI data tooling

File: `convert_patient_vcf.py` — argparse CLI converting between CSV and VCF and generating test data; expects INFO strings like `GENE=BRAF;MUT=V600E`.

- **CSV → VCF** `csv_to_patient_vcf`: validates required columns CHROM, POS, REF, ALT, GENE, MUT (raises listing the missing ones); emits VCFv4.2 with variant IDs `chrom:pos:ref>alt`, PASS filters, and `GENE=/MUT= INFO; MUTATION` accepted as a MUT alias, falling back to `ref>alt` — `convert_patient_vcf.py`.
- **VCF → CSV** `vcf_to_patient_csv`: skips comments/malformed rows; recovers gene from `GENE|SYMBOL` and mutation from `MUT|HGVS_P|HGVS_P`, else `ref>alt`; writes a six-column CSV — `convert_patient_vcf.py`.
- **Demo generator** `--generate-demo --demo-count N`: requires the bootstrapped KB; samples DISTINCT gene/mutation pairs (errors if the KB has fewer than requested); fabricates deterministic pseudo-coordinates (chromosome cycling 1–22, arithmetic positions, alternating A>T / C>G / G>A alleles) producing a valid, fully matchable patient VCF — `convert_patient_vcf.py`.
- **Synthetic clinical generator** `--synthetic-clinical --synthetic-count N --seed S`: seeded shuffle (reproducible) over DB pairs with wraparound; writes self-describing headers `##synthetic_data=true, ##synthetic_coordinates=true, ##source=...`; enriches INFO with `EVIDENCE_TIER/SOURCE/SYNTHETIC=1` and SYNTH-prefixed IDs so downstream consumers can detect simulation — `convert_patient_vcf.py`.

- **IndiGen converter** `--indigen [--max-rows N]`: ingests raw unannotated population-genomics VCFs, preserving original coordinates/IDs while assigning fallback `GENE=UNKNOWN` and `MUT=REF>ALT` so the app can ingest every row — `convert_patient_vcf.py`.
- Dispatch logic chooses flows by flags first, then by input/output suffixes, erroring clearly on unsupported combinations — `convert_patient_vcf.py`.

Sample fixtures at the repo root exercise these flows: `test_patient.csv`, `tmp_patient.csv` (BOM-prefixed UTF-8), `sample_indigen_like.csv`.

7 Test suite

Directory: `tests/` — unittest-style classes executed with `pytest` (`make test`).

- **Auto-provisioning fixture**: `conftest.py` defines a session-scoped autouse `ensure_database` fixture creating a minimal four-row KB (BRAF, EGFR, KRAS, ERBB2) when `clinical_kb.db` is absent, so the suite passes without bootstrapping; also provides a `fixtures_dir` fixture — `tests/conftest.py`.
- **Clinical workflow tests** `ClinicalWorkflowTests`: exact match (EGFR L858R); prefix stripping (p.L858R still exact); gene-context never classified exact; unknown gene returns none with the “No Direct Match” sentinel; VCF quality report validated end-to-end against the committed fixture (2 variants, valid headers, zero skips, 100% coverage) — `tests/test_clinical_workflow.py`, fixture `tests/unmatched_test.vcf` (BRCA1 V1838E, TP53 R273H).
- **Cohort parsing tests** `CohortParsingTests`: cross-patient recurrence kept (two SAMPLE-tagged identical hotspots → 0 duplicates, 2 patients observed); same-patient repeat counted as the sole true duplicate; `unique_variant_combinations` counting — `tests/test_cohort_parsing.py`.

8 Developer tooling & infrastructure

8.1 Make targets

File: `Makefile`.

`help`, `install`, `install-dev` (dev deps + pre-commit hooks), `bootstrap` (runs `python -m backend.app.db.bootstrap`), `test`, `test-coverage`, `lint`, `format`, `typecheck` (mypy over backend/), `clean` (caches only, DB preserved), `clean-db` (deletes the KB), `clean-all`, `run` (via `run.sh`), `backend`, `frontend`, `demo` (generates `demo.vcf`), `all` (`install-dev` + `bootstrap` + `test`).

8.2 Launch scripts

- **run.sh**: bash launcher with `set -euo pipefail`; configurable `--backend-host/--backend-port/--frontend-port/--skip-install`; auto-creates `.venv` and installs requirements; starts `uvicorn` (`app.main:app` with `--app-dir backend`) and headless `Streamlit` in the background; `EXIT`-trap cleanup stops both processes — `run.sh`.
- **run.ps1**: PowerShell equivalent using `Start-Process -PassThru`, parameterized `host/ports/-SkipInstall`, stopping both services in `finally` — `run.ps1`.

8.3 Container support

- **Dockerfile**: `python:3.11-slim` base; installs `curl` + `gcc`; installs production deps; copies code; performs a best-effort KB bootstrap at build time (failure tolerated, retried at runtime); exposes `8000/8501`; `HEALTHCHECK` curls `/health` (30 s interval, 60 s start period); entrypoint `./run.sh` running both services in one container — `Dockerfile`.
- **docker-compose.yml**: two services from the same image — `backend` (`uvicorn` bound `0.0.0.0:8000`, `./backend/data` volume mount persisting the KB, `healthcheck`) and `frontend` (`Streamlit` `8501` with `PHARMAGEN_API_URL=http://backend:8000`, gated on backend health via `depends_on: condition: service_healthy`) — `docker-compose.yml`.

8.4 Quality gates

- **pre-commit hooks** `.pre-commit-config.yaml`: `ruff` lint with `autofix` + `ruff-format`; trailing-whitespace, end-of-file-fixer, YAML check, 1 MB large-file guard, merge-conflict detector, debug-statements detector. No CI pipeline exists — hooks are the enforcement gate.
- **Tool configuration** `pyproject.toml`:
 - `pytest`: `testpaths` tests, verbose short-`traceback` defaults.
 - `ruff`: target `py39`, line length 100, rules `E/W/F/I/B/UP/SIM`; ignores `E501` (formatter-owned) and `B008` (FastAPI dependency defaults); asserts allowed in tests (`S101` per-file ignore).
 - `mypy`: `py3.9` semantics, `warn_return_any`, missing imports ignored, tests excluded.

- coverage: measures backend/, omits tests and the CLI script, fails below 70%.
- Dependencies requirements.txt / requirements-dev.txt: FastAPI, uvicorn[standard], Streamlit, pandas, networkx, pyvis, requests, python-dotenv, reportlab, python-multipart; dev additions pytest, pytest-cov, ruff, mypy, pre-commit.

8.5 Repository hygiene files

- .env.example: template with provider/key/model/URL placeholders and commented PHARMAGEN_ONCOKB_FILE guidance.
- .gitignore: keeps generated artifacts out of git — the KB DB, downloaded TSVs, all *.vcf/*.csv except committed fixtures and the seed panel, caches, .env, archives, and the large GENIE file.
- .dockerignore: slim build context excluding git metadata, venvs, secrets, markdown except README, caches, and test trees.
- .editorconfig: 4-space indentation, LF endings, UTF-8; YAML 2-space; Makefile tabs; no whitespace trimming in Markdown.

9 Environment variable reference

Variable	Purpose	Default
PHARMAGEN_API_URL	Frontend API base override	auto-detect http://127.0.0.1:8000
PHARMAGEN_ONCOKB_FILE	Path to full OncoKB GENIE TSV	auto-detect repo root / tests/

10 Cross-cutting policies

- **Match policy:** exact (gene + mutation) is the only actionable class shown as treatment recommendations; gene_context is displayed separately with warnings; none receives no recommendation. Enforced in the matcher, the reports, and the UI.
- **Safety posture:** privacy rules forbid patient identifiers in logs and session state; synthetic datasets are labeled end-to-end (generator headers → API detection → UI banner).
- **Graceful degradation chain:** CIViC offline → 4-record panel; OncoKB GENIE absent → seed CSV; DB missing → tests self-provision.